Quantum Key Distribution (BB84)

Hackathon Project: Full-Stack Secure Cryptography Pipeline

Mahazoasy Dylane and Richard Watry

October 25, 2026

Executive Summary: The QKD Pipeline

Goal: Establish a perfectly secure, mathematically unbreakable cryptographic key between Alice (Server) and Bob (Client) over a noisy channel.

- **Phase 1: Transmission & Sifting** - Exchanging photons and filtering out basis mismatches.
- **Phase 2: Noise Estimation** - Detecting eavesdroppers using statistical bounds.
- **Phase 3: Cascade Error Correction** - Fixing natural noise via parity checks and binary search.
- **Phase 4: Privacy Amplification** - Squeezing out Eve's partial knowledge using memory-optimized Toeplitz hashing.

Phase 1: Transmission & Sifting

The Physics Concept:

- Alice generates 200,000 bits and encodes them using random bases (filters).
- We simulate **2% natural fiber-optic noise**.
- Bob measures with his own random bases. Over a public channel, they compare *only the bases*, discarding mismatched bits.

Engineering Flex: Constant-Time Indexing

Instead of losing track of original photon positions when the array shrinks, we map sifted keys back to original indices:

```
match_mask = (alice_bases == bob_bases)
sifted_key = bobs_raw_bits[match_mask]
raw_indices = np.where(match_mask)[0] # Tracks original photon
```

Phase 2: Noise Estimation (Eavesdropper Detection)

The Concept: Eve cannot measure a quantum state without altering it. We sample 15k bits to audit the channel.

- **The "Burned Bits" Principle:** Any bit revealed over the API to calculate the error rate is mathematically compromised and permanently deleted from the key.
- **The Hoeffding Bound (Q_u):** We don't rely on raw averages (Q_{hat}). We calculate the worst-case error rate with a $1 - 10^{-10}$ confidence interval.

The Shor-Preskill Limit

If $Q_u > 11\%$, Eve knows too much. The protocol securely aborts. If $< 11\%$, we proceed.

Phase 3: Cascade Error Correction

The Goal: Fix the remaining 2% natural noise without revealing the key.

- **Binary Search Parity:** Bob breaks the key into blocks. If Alice and Bob's parities mismatch, an error exists. Bob splits the block in half recursively until the exact flipped bit is found.
- **Permutations:** The key is shuffled 3 times to break up hidden pairs of errors that cancel out parity checks.

Engineering Flex: CPU-Native Parity

To prevent integer memory overflows when summing 50,000 bits, parity is calculated via native logic gates:

```
bob_parity = np.bitwise_xor.reduce(secure_key[work_block])
```

Phase 4: Privacy Amplification

The Threat: Eve gained information from quantum noise, plus every parity bit leaked during the Cascade API calls. **The Math:** We shrink the key from length

n to l to squeeze Eve's knowledge to exactly zero.

$$l = n - (n \times h(Q_u)) - \text{leak}_{EC} - 2\log_2\left(\frac{1}{\epsilon_{PA}}\right)$$

- $h(Q_u)$: Binary entropy (Eve's quantum knowledge).
- leak_{EC}: The exact count of parity bits sent over the API.

We hash the key by multiplying it by a Toeplitz Matrix.

Technical Highlight: Zero-RAM Virtual Matrix

The Problem: A Toeplitz matrix for a 100,000-bit key requires 4 billion elements, causing a guaranteed RAM crash (OOM).

The Solution: We used a 1D seed and Numpy's `sliding_window_view` to project a virtual 2D matrix that uses **0 bytes of extra RAM**.

```
# Virtual 2D view over a 1D array
T_view = sliding_window_view(base_array, n)[::-1]

# Chunked Galois Field 2 Multiplication directly on CPU
for i in range(0, l, chunk_size):
    T_chunk = T_view[i:i+chunk_size]
    final_key[i:i+chunk_size] = np.bitwise_xor.reduce(
        T_chunk & secure_key, axis=1
    )
```

Appendix: 4-Bit Privacy Amplification Example

If asked how Privacy Amplification works on a small scale:

- **Original Key ($n = 4$):** `[1, 0, 1, 1]`
- **Target Length ($l = 3$)**
- **Seed Array (6 bits):** `1, 0, 1, 1, 0, 0`

The sliding window magically creates the Toeplitz Matrix:

Row 0: `[1, 1, 0, 0]`
Row 1: `[0, 1, 1, 0]`
Row 2: `[1, 0, 1, 1]`

GF(2) Multiplication (AND then XOR) scrambles the bits perfectly:

- Row 0 $\times$ Key $= 1 \oplus 0 \oplus 0 \oplus 0 = \mathbf{1}$
- Final Secure Key = `[1, 1, 1]`

Thank You

Result: A 100% provably secure, memory-safe, real-time cryptographic key distribution system.